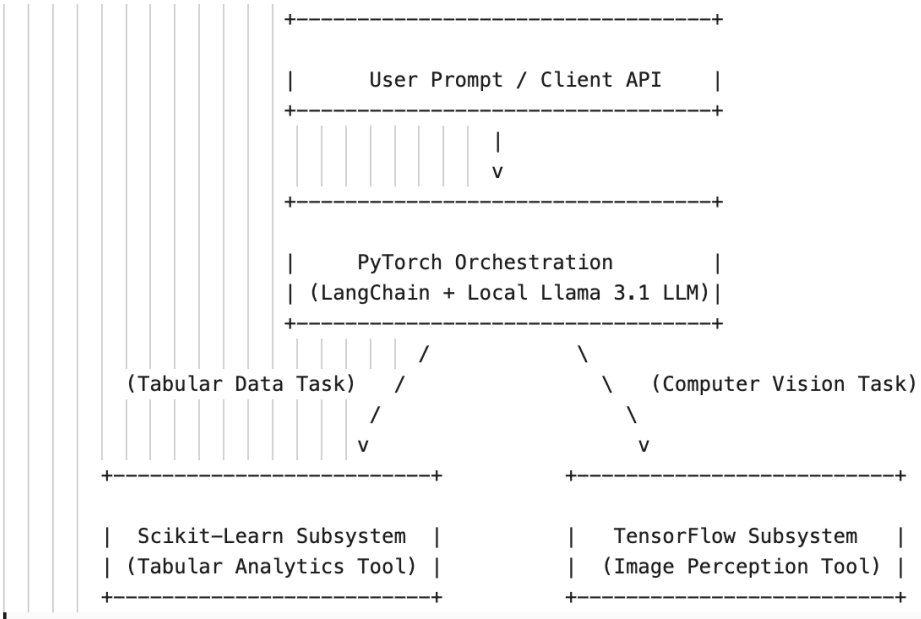
OmniMind Multi-Framework AI Agent Ecosystem

OmniMind: Multi-Framework AI Agent Ecosystem

An enterprise-grade, locally deployed AI Agent architecture that unifies three distinct machine learning paradigms. The system leverages an asynchronous orchestration engine to route tasks to framework-specific specialized subsystems based on the type of incoming data.

System Architecture



Architectural Breakdown

1. Orchestration Layer (PyTorch): Utilizes a local LLM via Hugging Face/Ollama. This layer processes natural language intents, manages state, and dynamically selects tools using function-calling mechanisms.
2. Analytics Layer (Scikit-learn): A highly efficient, CPU-bound pipeline optimized for memory-mapped structured datasets. It provides deterministic, rapid inference for classification and regression tasks without deep learning overhead.
3. Perception Layer (TensorFlow): A parallelized, GPU-optimized deep convolutional pipeline designed for unstructured computer vision payloads.

OmniMind Project

OmniMind is a local, multi-framework AI agent system that routes user requests to specialized tools:

- Tabular analytics with Scikit-learn
- Vision inference with TensorFlow (MobileNetV2 workflow)
- Agent orchestration with LangChain + Ollama

Technology Stack

- Language: Python 3
- Agent Runtime: LangChain, LangChain Ollama
- Local LLM Provider: Ollama (configured model: llama3.1)
- Tabular ML: Scikit-learn, Joblib, NumPy
- Vision ML: TensorFlow / Keras (MobileNetV2 transfer learning)
- Image IO: Pillow
- Environment: venv

Project Structure (With Comments)

```text
omnimind-project/
├── config.py                    # Central paths and runtime model configuration
├── requirements.txt             # Python dependencies
├── train_artifacts.py           # Main artifact/training pipeline (sklearn + vision fine-tune)
├── train_artifacts_mock.py      # Alternate/mock training script variant
├── README.md                    # Project documentation
|
├── data/
|   ├── sample_image.jpeg        # Default sample image path used by orchestrator
|   ├── sample_image_dog.jpg     # Sample dog image (dataset/bootstrap support)
|   ├── sample_image_dots.jpg    # Sample dots image (dataset/bootstrap support)
|   ├── models/
|   |   ├── housing_model.joblib  # Serialized Scikit-learn tabular model artifact
|   |   ├── vision_model.keras    # Serialized TensorFlow vision model artifact
|   |   └── vision_labels.json    # Label metadata for custom fine-tuned classes
|   └── vision/
|       ├── README.md             # Vision dataset format and naming rules
|       ├── train/                # Labeled training images by class folder
|       └── val/                  # Labeled validation images by class folder
|
└── src/
    ├── __init__.py             # Package exports for src
```

```
    ├── orchestrator.py            # LangChain/Ollama agent that routes to tools
    └── tools/
        ├── __init__.py            # Tool package exports
        ├── analytics.py           # Real-estate valuation tool (tabular regression)
        ├── perception.py          # Vision inference tool (top-k predictions)
        └── perception_mock.py     # Alternate/mock vision tool implementation
```

1. Configuration Module (config.py)
This file ensures absolute paths and environmental settings are unified across all frameworks.
2. Mock Training Script (train_artifacts.py)
This script generates a valid scikit-learn linear regressor and a TensorFlow Convolutional Neural Network (CNN), serializing them to disk so the agent has functional tools immediately.
3. Analytics Tool Layer (src/tools/analytics.py)
This module manages the Scikit-learn sub-engine. It includes structural error handling to verify the presence of model binaries before running standard CPU metrics.
4. Perception Tool Layer (src/tools/perception.py)
This module manages the TensorFlow sub-engine, wrapping native array preprocessing operations inside safe execution bounds.
5. Orchestration Layer (src/orchestrator.py)
This module uses a PyTorch-backed language model framework (langchain interfacing with local Ollama models) to bind the custom tools and route work dynamically based on incoming intent patterns.

File-by-File Responsibilities

Root Files

- config.py
  - Defines canonical paths for models, datasets, and sample images
  - Defines LOCAL_LLM_MODEL used by the orchestrator

- requirements.txt
  - Lists all required Python libraries for orchestration, ML, and image handling

- train_artifacts.py
  - Trains and saves the Scikit-learn housing model
  - Builds/fine-tunes TensorFlow MobileNetV2 when a valid dataset exists
  - Validates dataset quality (class alignment, minimum distinct images, no train/val leakage)
  - Saves vision labels metadata and prints per-class validation accuracy

- train_artifacts_mock.py
  - Mock/alternate training path used for experimentation or legacy behavior

- README.md
  - Main project documentation (this file)

Source Package

- src/__init__.py
  - Exposes package-level imports for the src module

- src/orchestrator.py
  - Initializes ChatOllama model and LangChain agent
  - Registers analytics and vision tools
  - Routes user prompts to the correct subsystem

- src/tools/__init__.py
  - Re-exports tool entry points for clean imports

- src/tools/analytics.py
  - Loads tabular model artifact
  - Parses tool input in square_foot,bedrooms format
  - Returns formatted valuation output

- src/tools/perception.py
  - Loads TensorFlow model artifact and optional custom labels
  - Preprocesses input image and performs inference
  - Returns top prediction and top-3 confidence output

- src/tools/perception_mock.py
  - Mock/alternate vision behavior for testing

Data and Artifacts

- data/models/housing_model.joblib
  - Trained tabular model used by analytics tool

- data/models/vision_model.keras
  - Trained or fallback vision model used by perception tool

- data/models/vision_labels.json
  - Indicates custom vs imagenet label source and class names

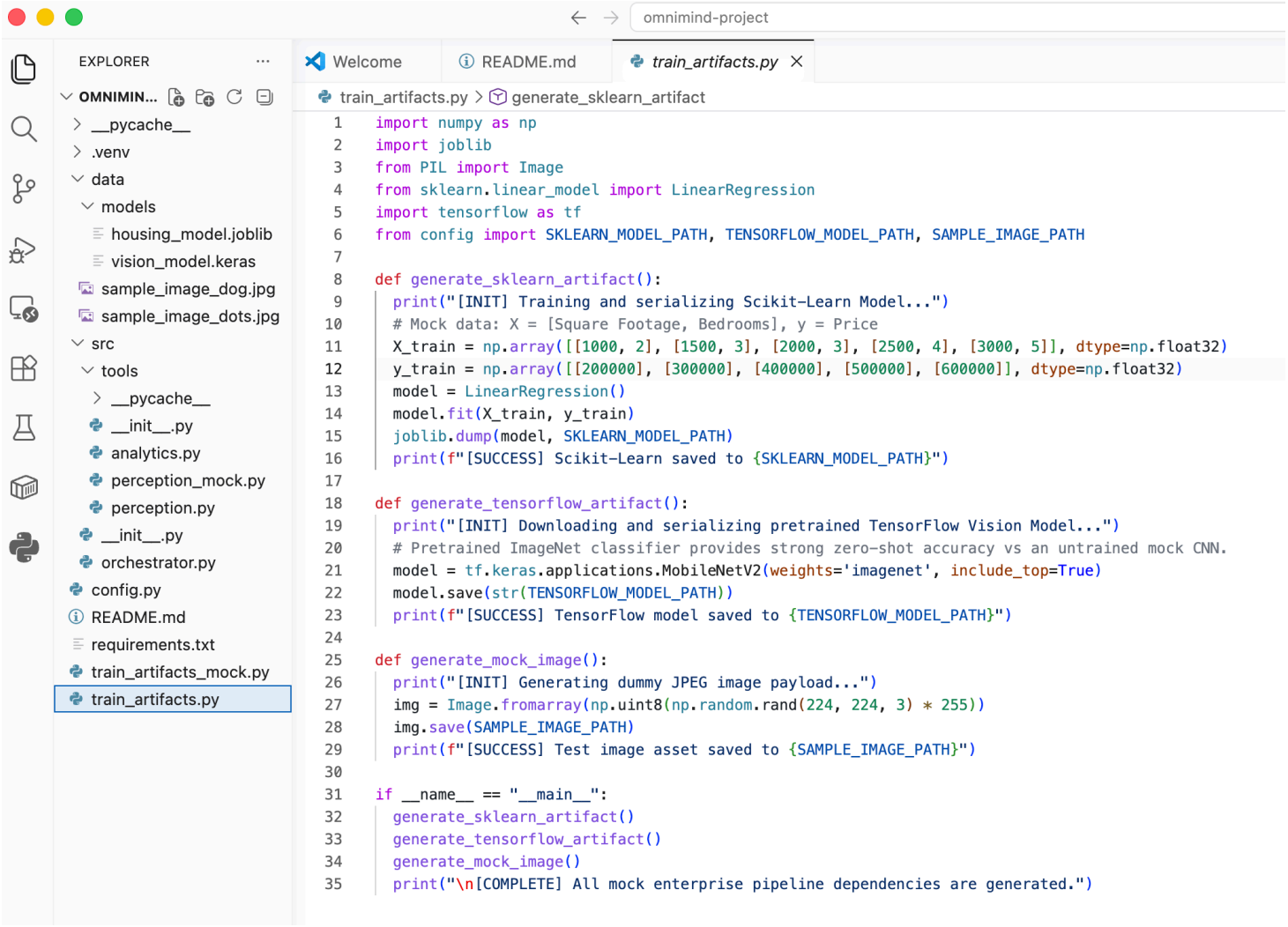
- data/vision/train and data/vision/val
  - Class-folder datasets used for fine-tuning and validation

- data/vision/README.md
  - Dataset layout, naming conventions, and usage notes

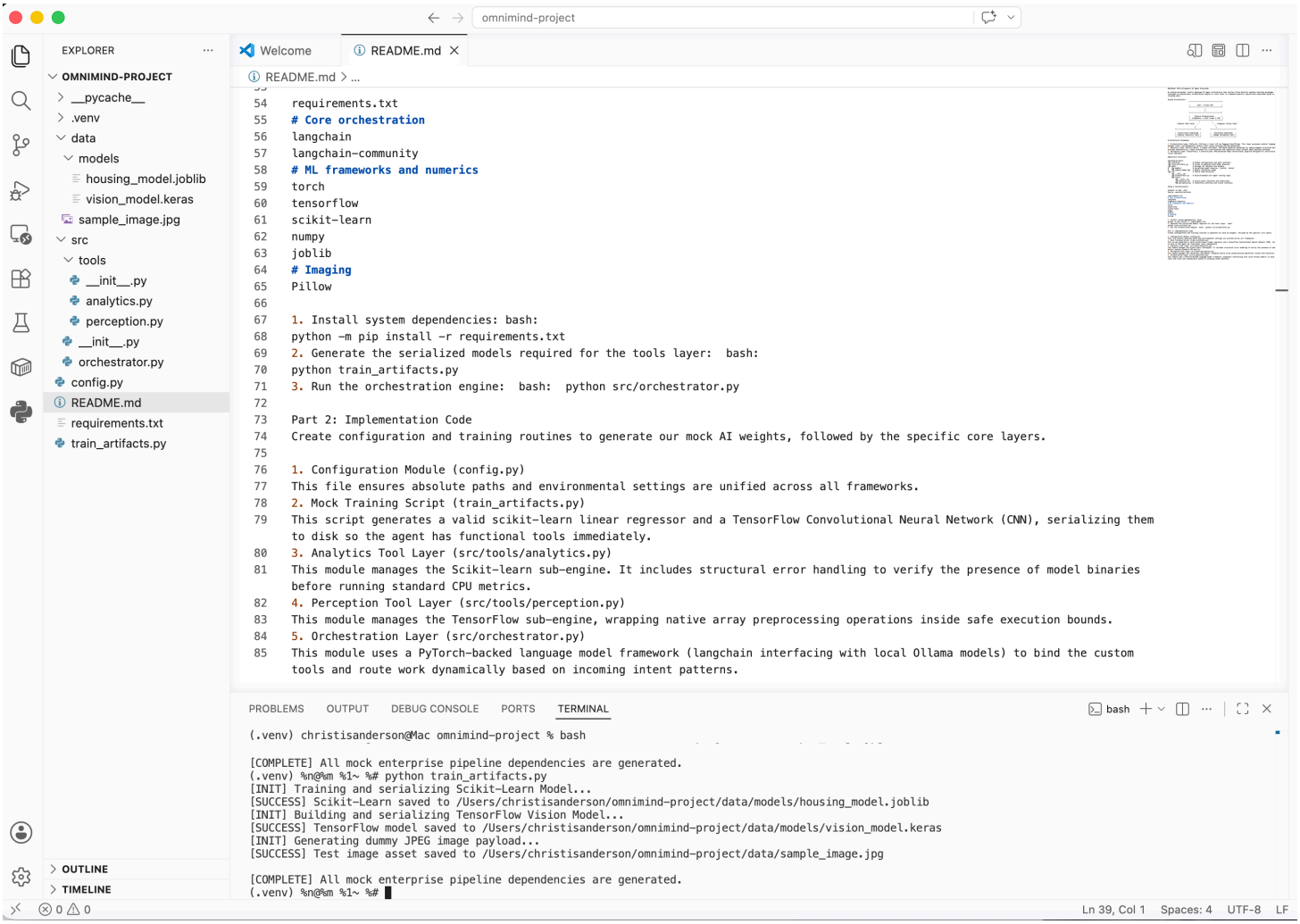
How the System Works (Summary)

1. You generate artifacts with train_artifacts.py.
2. The orchestrator starts and loads the local Ollama LLM.
3. User prompt enters the orchestrator.
4. Agent selects analytics or perception tool.
5. Tool loads its model artifact and returns a prediction.

Here is the training artifacts code that seeds the analytics tool (Scikit-Learn Model), seeds the perception tool (TensorFlow Vision Model), and creates a default test image (dot pixels):



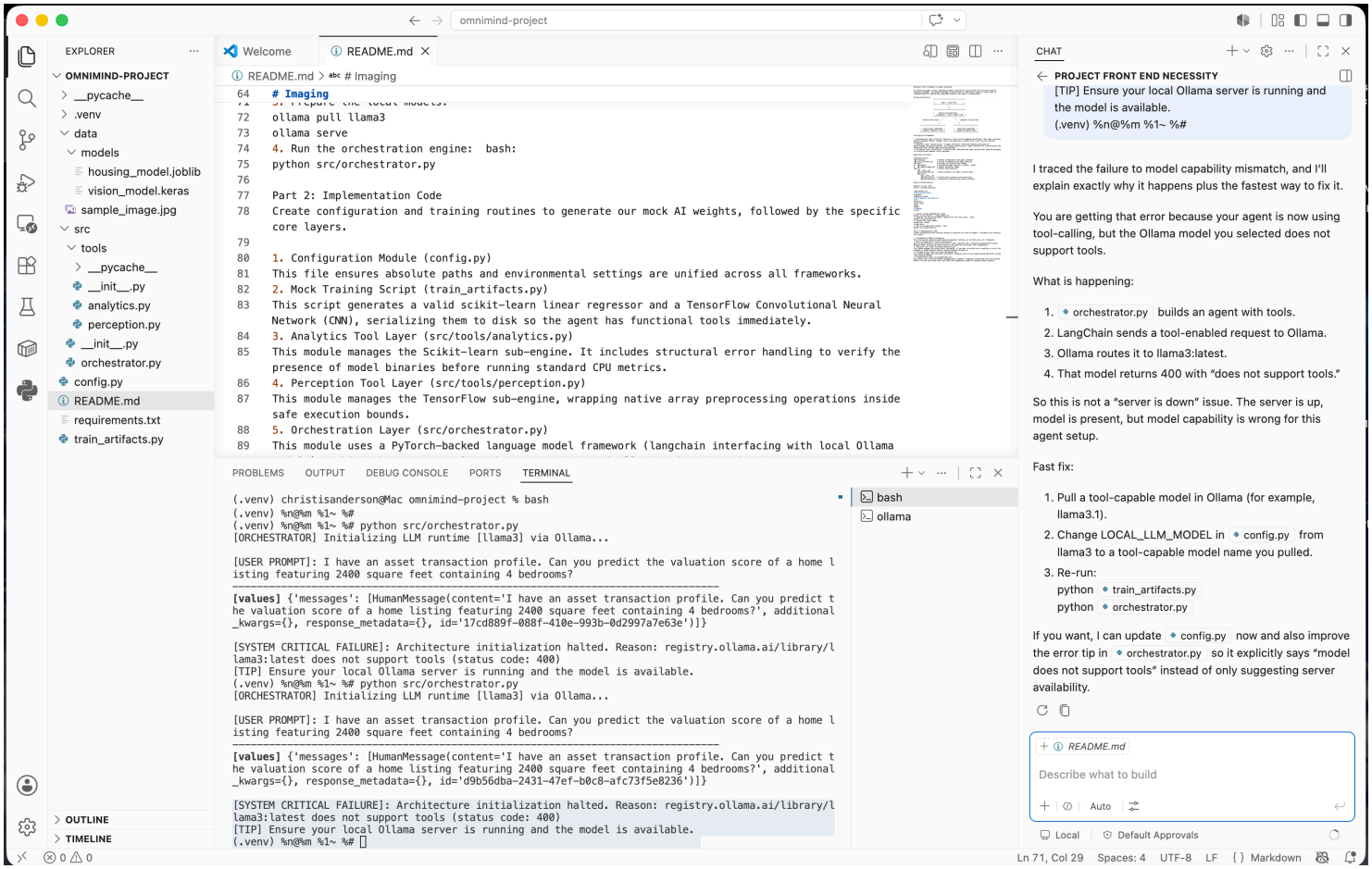
Generate the serialized models required for the tools layer



Start the Orchestrator
Example of SYSTEM_CRITICAL_FAILURE (need to start Ollama server—model not available)The

```
(.venv) %n@%m %1~ %#
(.venv) %n@%m %1~ %# python src/orchestrator.py
[ORCHESTRATOR] Initializing LLM runtime [llama3] via Ollama...

[USER PROMPT]: I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?
--------------------------------------------------------------------------------
[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square
feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='17cd889f-088f-410e-993b-0d2997a7e63e')]}

[SYSTEM CRITICAL FAILURE]: Architecture initialization halted. Reason: registry.ollama.ai/library/llama3:latest does not support tools (status code: 400)
[TIP] Ensure your local Ollama server is running and the model is available.
(.venv) %n@%m %1~ %#
```

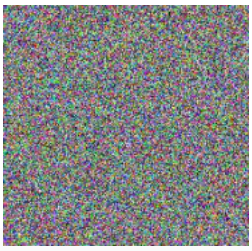
Orchestrator agent is using tool-calling, but llama3 does not support tools. Using GitHub Copilot to help resolve the error (switch to llama3.1).



Now that llama3.1 is running successfully start the testing.

For this prompt, Orchestrator routed to Tabular Analytics Tool and gave proper answer based on seeded train data:

For this prompt, Orchestrator routed to Image Perception Tool and gave proper answer based on seeded train data (dot image):



```
[USER PROMPT]: We have ingested an unstructured binary media payload file. Please audit what is ins
ide the asset located at file path: /Users/christisanderson/omnimind-project/data/sample_image.jpg
--------------------------------------------------------------------------------
[values] {'messages': [HumanMessage(content='We have ingested an unstructured binary media payload
file. Please audit what is inside the asset located at file path: /Users/christisanderson/omnimind-
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

bash
ollama

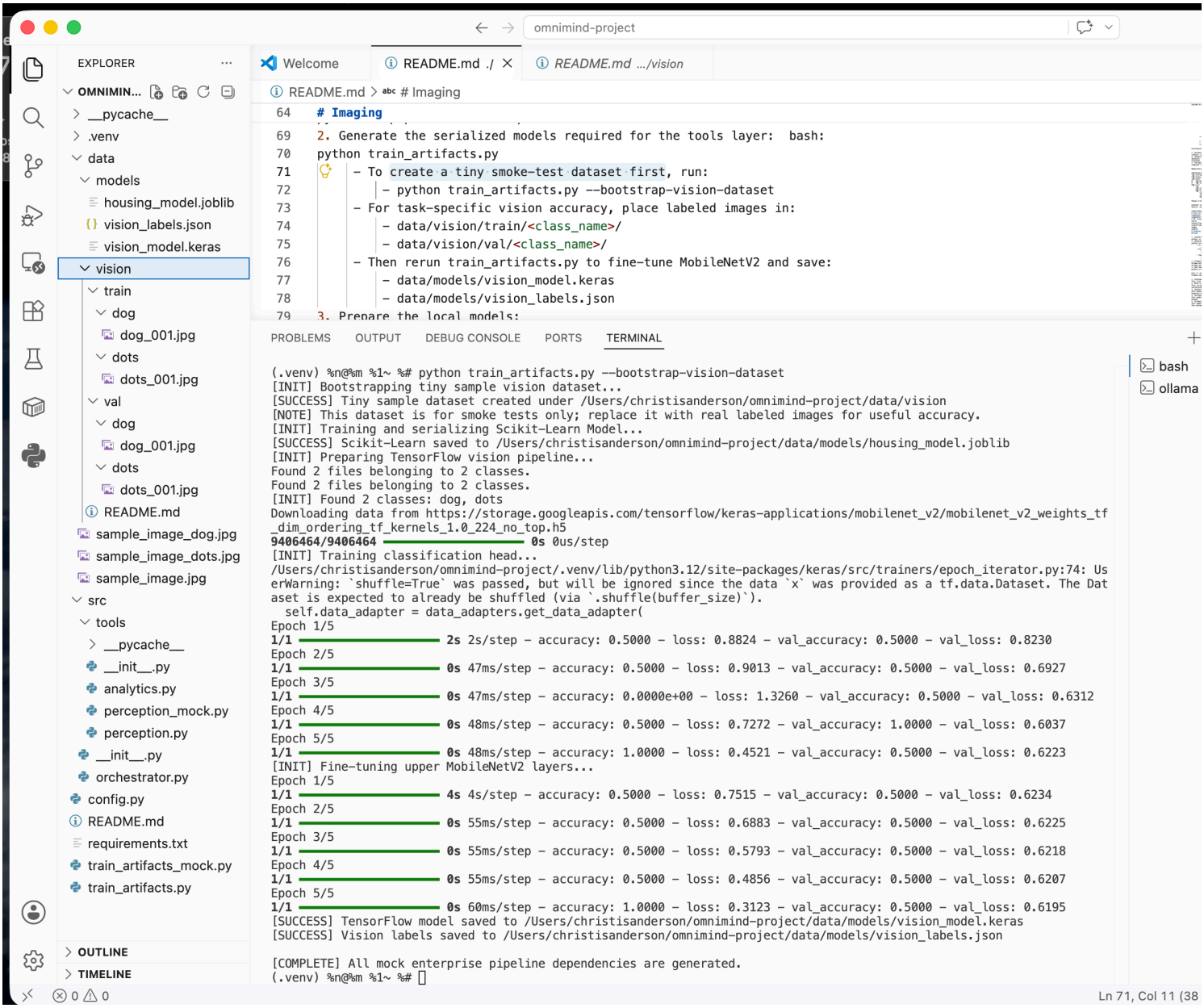
```
'model_provider': 'ollama'}, id='lc_run--019f1bd6-9903-7d21-959c-325f056c8be5-0', tool_calls=[{'nam
e': 'run_vision_inference', 'args': {'image_path': '/Users/christisanderson/omnimind-project/data/s
ample_image.jpg'}, 'id': 'cb182516-b4eb-4891-a9bd-95dbfc7f706a', 'type': 'tool_call'}], invalid_too
l_calls=[], usage_metadata={'input_tokens': 316, 'output_tokens': 35, 'total_tokens': 351}), ToolMe
ssage(content="TensorFlow Inference Result: Top prediction -> 'velvet' with 42.92% confidence. Top-
3: velvet (42.92%), wool (15.53%), bubble (10.02%).", name='run_vision_inference', id='d3cadfbc-001
0-41b3-be04-00203adb33e3', tool_call_id='cb182516-b4eb-4891-a9bd-95dbfc7f706a')]}
[updates] {'model': {'messages': [AIMessage(content='Based on the vision tool\'s output, it appears
 that the image at the specified file path is a picture of velvet fabric. The top prediction from t
he model is "velvet" with a confidence level of 42.92%. The top three predictions are:\n\n1. Velvet
 (42.92%)\n2. Wool (15.53%)\n3. Bubble (10.02%)\n\nThis suggests that the image contains characteri
stics commonly associated with velvet, such as its texture and appearance.', additional_kwargs={},
response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T04:01:24.948403Z', 'done': True,
'done_reason': 'stop', 'total_duration': 5390821166, 'load_duration': 114129458, 'prompt_eval_count
': 210, 'prompt_eval_duration': 794511000, 'eval_count': 96, 'eval_duration': 4475707000, 'logprobs
': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bd6-a4c4-7f82-98f2
-ec0536364ab7-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 210, 'outpu
t_tokens': 96, 'total_tokens': 306})]}}
[values] {'messages': [HumanMessage(content='We have ingested an unstructured binary media payload
file. Please audit what is inside the asset located at file path: /Users/christisanderson/omnimind-
project/data/sample_image.jpg', additional_kwargs={}, response_metadata={}, id='ed7f30f6-005d-44b7-
b56e-618a47a288da'), AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'llama
3.1', 'created_at': '2026-07-01T04:01:18.583357Z', 'done': True, 'done_reason': 'stop', 'total_dura
tion': 2035095292, 'load_duration': 99587500, 'prompt_eval_count': 316, 'prompt_eval_duration': 322
600000, 'eval_count': 35, 'eval_duration': 1606920000, 'logprobs': None, 'model_name': 'llama3.1',
'model_provider': 'ollama'}, id='lc_run--019f1bd6-9903-7d21-959c-325f056c8be5-0', tool_calls=[{'nam
e': 'run_vision_inference', 'args': {'image_path': '/Users/christisanderson/omnimind-project/data/s
ample_image.jpg'}, 'id': 'cb182516-b4eb-4891-a9bd-95dbfc7f706a', 'type': 'tool_call'}], invalid_too
l_calls=[], usage_metadata={'input_tokens': 316, 'output_tokens': 35, 'total_tokens': 351}), ToolMe
ssage(content="TensorFlow Inference Result: Top prediction -> 'velvet' with 42.92% confidence. Top-
3: velvet (42.92%), wool (15.53%), bubble (10.02%).", name='run_vision_inference', id='d3cadfbc-001
0-41b3-be04-00203adb33e3', tool_call_id='cb182516-b4eb-4891-a9bd-95dbfc7f706a'), AIMessage(content=
'Based on the vision tool\'s output, it appears that the image at the specified file path is a pict
ure of velvet fabric. The top prediction from the model is "velvet" with a confidence level of 42.9
2%. The top three predictions are:\n\n1. Velvet (42.92%)\n2. Wool (15.53%)\n3. Bubble (10.02%)\n\nT
his suggests that the image contains characteristics commonly associated with velvet, such as its t
exture and appearance.', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at'
: '2026-07-01T04:01:24.948403Z', 'done': True, 'done_reason': 'stop', 'total_duration': 5390821166,
 'load_duration': 114129458, 'prompt_eval_count': 210, 'prompt_eval_duration': 794511000, 'eval_cou
nt': 96, 'eval_duration': 4475707000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider':
 'ollama'}, id='lc_run--019f1bd6-a4c4-7f82-98f2-ec0536364ab7-0', tool_calls=[], invalid_tool_calls=
[], usage_metadata={'input_tokens': 210, 'output_tokens': 96, 'total_tokens': 306})]}
--------------------------------------------------------------------------------
[AGENT FINAL RESPONSE]:
Based on the vision tool's output, it appears that the image at the specified file path is a pictur
e of velvet fabric. The top prediction from the model is "velvet" with a confidence level of 42.92%
. The top three predictions are:

1. Velvet (42.92%)
2. Wool (15.53%)
3. Bubble (10.02%)

This suggests that the image contains characteristics commonly associated with velvet, such as its
texture and appearance.

(.venv) %n@%m %1~ %#
```

For this prompt, Orchestrator routed to Image Perception Tool and gave proper answer based on seeded train data (dog image):


```
.", additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T04:35:1
4.991927Z', 'done': True, 'done_reason': 'stop', 'total_duration': 6777721750, 'load_duration': 974
61791, 'prompt_eval_count': 216, 'prompt_eval_duration': 792856000, 'eval_count': 126, 'eval_durati
on': 5879021000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_ru
n--019f1bf5-9935-7550-a0b8-d210e31e3a7b-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'
input_tokens': 216, 'output_tokens': 126, 'total_tokens': 342})]}
--------------------------------------------------------------------------------
[AGENT FINAL RESPONSE]:
Based on the output of the vision tool, it appears that the image at the specified file path contai
ns a picture of a golden retriever. The top prediction is 'golden retriever' with 56.02% confidence
, indicating that this is the most likely object in the image. The top-3 predictions also include a
n otterhound and a clumber, but these are less likely objects in the image.

Therefore, the answer to your original question is:

The asset located at file path: /Users/christisanderson/omnimind-project/data/sample_image.jpg cont
ains a picture of a golden retriever.

(.venv) %n@%m %1~ %# $
```

Now do MobileNetV2 fine-tuning instead of only generic ImageNet inference when there are labeled images found in class folders.

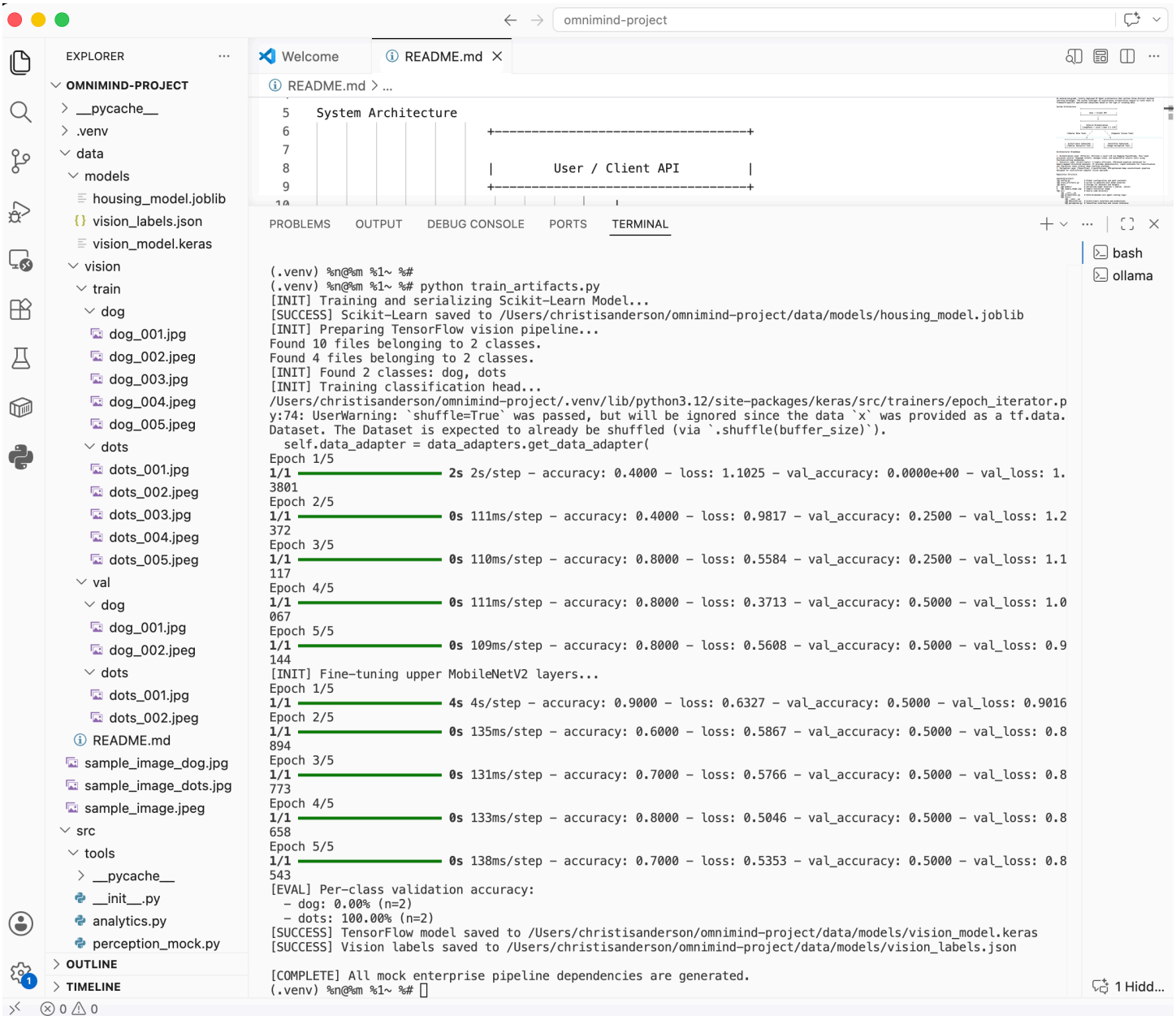
First, do a quick POC by running a bootstrap vision dataset loader to load a tiny smoke-test dataset structure:



Added error to inform for the need to add required minimum number of unique images for each class: ValueError: Vision dataset validation failed. Fine-tuning was skipped because the dataset does not meet minimum distinct-image requirements

```
(.venv) %n@%m %1~ %# python train_artifacts.py
[INIT] Training and serializing Scikit-Learn Model...
[SUCCESS] Scikit-Learn saved to /Users/christisanderson/omnimind-project/data/models/housing_model.joblib
[INIT] Preparing TensorFlow vision pipeline...
Traceback (most recent call last):
  File "/Users/christisanderson/omnimind-project/train_artifacts.py", line 249, in <module>
    generate_tensorflow_artifact()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/christisanderson/omnimind-project/train_artifacts.py", line 162, in generate_tensorflow_artifact
    _validate_vision_dataset()
  File "/Users/christisanderson/omnimind-project/train_artifacts.py", line 152, in _validate_vision_dataset
    raise ValueError(
ValueError: Vision dataset validation failed. Fine-tuning was skipped because the dataset does not meet minimum distinct-image requirements: class 'dog' has
 only 1 distinct training images; minimum is 5; class 'dog' has only 1 distinct validation images; minimum is 2; class 'dog' reuses 1 identical image(s) acr
oss train and val; class 'dots' has only 1 distinct training images; minimum is 5; class 'dots' has only 1 distinct validation images; minimum is 2; class '
dots' reuses 1 identical image(s) across train and val.
(.venv) %n@%m %1~ %#
```

Fixed



Model was able to successfully categorize the image after applying fine-tuning: